Q:- 1. Create a database named schoolDB and a collection students. Insert 5 student documents with fields: name, age, grade.
1. Write a query to fetch all students whose grade is A.
2. Write a query to insert multiple documents into a collection.
3. Write a query to update the age of a student where name = "John".
4. Write a query to delete a document where grade = "F".
5. Write a query to find all documents from a collection.

- **Input and Output :-**

```
test> use schoolDB
switched to db schoolDB
schoolDB> db.createCollection("students");
{ ok: 1 }
schoolDB> db.students.insertMany([{ name: "Palak", age: 16, grade: "A" },{ name: "John", age: 17, grade: "B" },{ name: "Ritu", age: 16, grade: "A" },{ name:
, grade: "F" }]);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ab00bde776215c5d16e1466'),
    '1': ObjectId('6ab00bde776215c5d16e1467'),
    '2': ObjectId('6ab00bde776215c5d16e1468'),
    '3': ObjectId('6ab00bde776215c5d16e1469'),
    '4': ObjectId('6ab00bde776215c5d16e146a')
  }
}
```

- **Input and output (i):-**

```
schoolDB> db.students.find({ grade: "A" });
[
  {
    _id: ObjectId('6ab00bde776215c5d16e1466'),
    name: 'Palak',
    age: 16,
    grade: 'A'
  },
  {
    _id: ObjectId('6ab00bde776215c5d16e1468'),
    name: 'Ritu',
    age: 16,
    grade: 'A'
  }
]
```

- **Input and output (ii):-**

```
schoolDB> db.students.insertMany([{ name: "Palak", age: 16, grade: "A" },{ name: "John", age: 17, grade: "B" },{ name: "Ritu", age: 16, grade: "A" },{ name:
, grade: "F" }]);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ab00bde776215c5d16e1466'),
    '1': ObjectId('6ab00bde776215c5d16e1467'),
    '2': ObjectId('6ab00bde776215c5d16e1468'),
    '3': ObjectId('6ab00bde776215c5d16e1469'),
    '4': ObjectId('6ab00bde776215c5d16e146a')
  }
}
```

- **Input and output (iii):-**

```
schoolDB> db.students.updateOne({ name: "John" },{ $set: { age: 17 } });
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 0,
  upsertedCount: 0
}
```

- **Input and output (iv):-**

```
schoolDB> db.students.deleteOne({ grade: "F" })
{ acknowledged: true, deletedCount: 1 }
```

- **Input and output (v):-**

```
schoolDB> db.students.find();
[
  {
    _id: ObjectId('6ab00bde776215c5d16e1466'),
    name: 'Palak',
    age: 16,
    grade: 'A'
  },
  {
    _id: ObjectId('6ab00bde776215c5d16e1467'),
    name: 'John',
    age: 17,
    grade: 'B'
  },
  {
    _id: ObjectId('6ab00bde776215c5d16e1468'),
    name: 'Ritu',
    age: 16,
    grade: 'A'
  },
  {
    _id: ObjectId('6ab00bde776215c5d16e1469'),
    name: 'Palki',
    age: 18,
    grade: 'C'
  }
]
```

Q:-2 Create an Employee schema with name, department, salary, and experience. Implement Create, Read, Update, and Delete (CRUD) operations using Node.js and Mongoose.

- **Input(i):-**
  - **Index.js:-**

```
const mongoose = require("mongoose");

// Connect to MongoDB
async function connectDB() {
    try {
        await mongoose.connect("mongodb://localhost:27017/crud");
        console.log("MongoDB connected");
    } catch (err) {
        console.error("Connection error:", err);
    }
}

// Define Employee model
const Employee = mongoose.model("Employee", {
    name: String,
    department: String,
    salary: Number,
    experience: Number
});

// Insert employees
async function createEmployees() {
    try {
        // Insert a single employee
        const newEmployee = new Employee({
            name: "Palki",
            department: "IT",
            salary: 50000,
            experience: 2
        });

        const savedEmployee = await newEmployee.save();
        console.log("Single employee saved:", savedEmployee);
```

```
        // Insert multiple employees
        const manyEmployees = await Employee.insertMany([
            {
                name: "Ritu",
                department: "HR",
                salary: 45000,
                experience: 3
            },
            {
                name: "Palu",
                department: "Finance",
                salary: 60000,
                experience: 5
            },
            {
                name: "Palak",
                department: "IT",
                salary: 55000,
                experience: 4
            }
        ]);
        console.log("Multiple employees inserted:", manyEmployees);

    } catch (err) {
        console.error("Error:", err);
    }
}

// Run the program
async function main() {
    await connectDB();
    await createEmployees();
}

main();
```

- **Output(i):-**

```
PS C:\Users\Admin\OneDrive\Desktop\Journal\Mean\p2> node index.js
MongoDB connected
Single employee saved: {
  name: 'Palki',
  department: 'IT',
  salary: 50000,
  experience: 2,
  _id: new ObjectId('6ab0e5041647788e299b43fc'),
  __v: 0
}
Multiple employees inserted: [
  {
    name: 'Ritu',
    department: 'HR',
    salary: 45000,
    experience: 3,
    _id: new ObjectId('6ab0e5041647788e299b43fd'),
    __v: 0
  },
  {
    name: 'Palu',
    department: 'Finance',
    salary: 60000,
    experience: 5,
    _id: new ObjectId('6ab0e5041647788e299b43fe'),
    __v: 0
  },
  {
    name: 'Palak',
    department: 'IT',
    salary: 55000,
    experience: 4,
    _id: new ObjectId('6ab0e5041647788e299b43ff'),
    __v: 0
  }
]
```

- **Input(ii):-**
  - **Find.js:-**

```
const mongoose = require("mongoose");

// Connect to MongoDB
async function connectDB() {
    try {
        await mongoose.connect("mongodb://localhost:27017/crud");
        console.log("Connected to MongoDB");
    } catch (err) {
        console.error("Connection error:", err);
    }
}
```

```
// Define Employee model
const Employee = mongoose.model("Employee", {
    name: String,
    department: String,
    salary: Number,
    experience: Number
});

// Find employees
async function findEmployees() {
    try {
        // Find one employee
        const employee = await Employee.findOne({
            name: "Palak"
        });

        if (employee) {
            console.log("Employee found:", employee);
        } else {
            console.log("No employee found");
        }

        // Find employees with salary >= 50000
        const employees = await Employee.find({
            salary: { $gte: 50000 }
        });

        if (employees.length > 0) {
            console.log("Employees found:");

            employees.forEach(employee => {
                console.log(employee);
            });
        } else {
            console.log("No employees found");
        }

    } catch (err) {
        console.error("Find error:", err);
    } finally {
```

```
        mongoose.connection.close();
    }
}

// Run the program
async function main() {
    await connectDB();
    await findEmployees();
}

main();
```

- **Output(ii):-**



```
PS C:\Users\Admin\OneDrive\Desktop\Journal\Mean\p2> node find.js
Connected to MongoDB
Employee found: {
  _id: new ObjectId('6ab0e5041647788e299b43ff'),
  name: 'Palak',
  department: 'IT',
  salary: 55000,
  experience: 4,
  __v: 0
}
Employees found:
{
  _id: new ObjectId('6ab0e5041647788e299b43fc'),
  name: 'Palki',
  department: 'IT',
  salary: 50000,
  experience: 2,
  __v: 0
}
{
  _id: new ObjectId('6ab0e5041647788e299b43fe'),
  name: 'Palu',
  department: 'Finance',
  salary: 60000,
  experience: 5,
  __v: 0
}
{
  _id: new ObjectId('6ab0e5041647788e299b43ff'),
  name: 'Palak',
  department: 'IT',
  salary: 55000,
  experience: 4,
  __v: 0
}
```

- **Input(iii)**:-
  - Update.js

```
const mongoose = require("mongoose");

// Connect to MongoDB
async function connectDB() {
    try {
        await mongoose.connect("mongodb://localhost:27017/crud");
        console.log("Connected to MongoDB");
    } catch (err) {
        console.error("Connection error:", err);
    }
}

// Define Employee model
const Employee = mongoose.model("Employee", {
    name: String,
    department: String,
    salary: Number,
    experience: Number
});

// Update employees
async function updateEmployees() {
    try {
        // Update one employee
        const result1 = await Employee.findOneAndUpdate(
            { name: "Palak" },          // Filter
            { salary: 60000 },          // Update
            { new: true }               // Return updated document
        );

        if (result1) {
            console.log("Employee updated:", result1);
        } else {
            console.log("No employee found");
        }

        // Update multiple employees
```

```
        const result = await Employee.updateMany(
            { experience: { $gte: 3 } },    // Filter
            { $set: { salary: 65000 } }     // Update
        );

        console.log(
            `Matched: ${result.matchedCount}, Modified: ${result.modifiedCount}`
        );

    } catch (err) {
        console.error("Update error:", err);
    }
}

// Run the program
async function main() {
    await connectDB();
    await updateEmployees();
}

main();
```

- **Output(iii):-**

```
PS C:\Users\Admin\OneDrive\Desktop\Journal\Mean\p2> node update.js
Connected to MongoDB
(node:10964) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is de
ecated. Use `returnDocument: 'after'` instead.
(Use `node --trace-warnings ...` to show where the warning was created)
Employee updated: {
  _id: new ObjectId('6ab0e5041647788e299b43ff'),
  name: 'Palak',
  department: 'IT',
  salary: 60000,
  experience: 4,
  __v: 0
}
Matched: 3, Modified: 3
```

- **Input(iv):-**
  - Delete.js:-

```
const mongoose = require("mongoose");

// Connect to MongoDB
async function connectDB() {
    try {
        await mongoose.connect("mongodb://localhost:27017/crud");
        console.log("Connected to MongoDB");
    } catch (err) {
        console.error("Connection error:", err);
    }
}

// Define Employee model
const Employee = mongoose.model("Employee", {
    name: String,
    department: String,
    salary: Number,
    experience: Number
});

// Delete employees
async function deleteEmployees() {
    try {
        // Delete one employee by name
        const deleteOneResult = await Employee.deleteOne({
            name: "Palak"
        });

        console.log(
            "Deleted one:",
            deleteOneResult.deletedCount
        );

        // Delete employees with experience <= 2
        const deleteManyResult = await Employee.deleteMany({
            experience: { $lte: 2 }
        });
```

```
        console.log(
            "Deleted many:",
            deleteManyResult.deletedCount
        );

    } catch (err) {
        console.error("Delete error:", err);
    }
}

// Run the program
async function main() {
    await connectDB();
    await deleteEmployees();
}

main();
```

- **Output(iv):-**

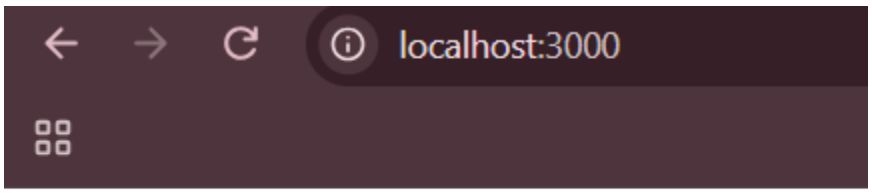
```
PS C:\Users\Admin\OneDrive\Desktop\Journal\Mean\p2> node delete.js
Connected to MongoDB
Deleted one: 1
Deleted many: 1
```

Q-3 Create an ExpressJS program to build a web server that displays “Welcome to ExpressJS” on the homepage.

- **Input:-**

```
const express = require("express");
const app = express();
app.get("/", (req, res) => {
res.send("Welcome to ExpressJS");
});
app.listen(3000, () => {
console.log("Server is running on http://localhost:3000");
});
```

- **Output:-**



Welcome to ExpressJS

Q-4 Create an ExpressJS program to configure routes for /, /about, and /contact pages with different responses.

- **Input:-**

```
const express = require("express");

const app = express();

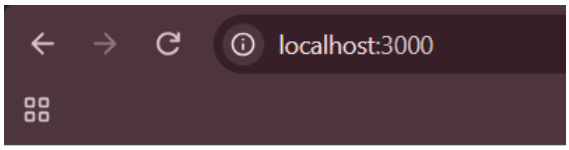
app.get("/", (req, res) => {
   res.send("Welcome to the Home Page");
});

app.get("/about", (req, res) => {
   res.send("This is the About Page");
});

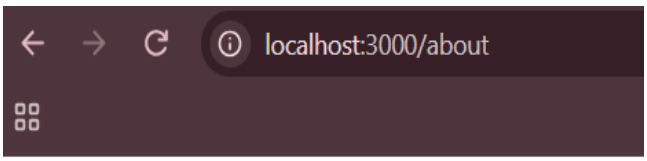
app.get("/contact", (req, res) => {
   res.send("This is the Contact Page");
});

app.listen(3000, () => {
   console.log("Server is running on http://localhost:3000");
});
```

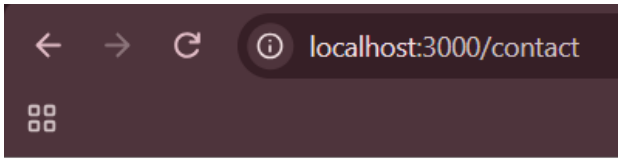
- **Output:-**



Welcome to the Home Page



This is the About Page



This is the Contact Page

Q-5 Create an ExpressJS program using middleware to log request details (method, URL, time) and parse JSON input.

- **Input:-**

```
const express = require("express");

const app = express();

// Parse JSON input
app.use(express.json());

// Middleware to log request details
app.use((req, res, next) => {
    const time = new Date().toLocaleString();

    console.log("Method:", req.method);
    console.log("URL:", req.url);
    console.log("Time:", time);
    console.log("--------------------------");

    next();
});

// Home route
app.get("/", (req, res) => {
    res.send("Welcome to ExpressJS");
});

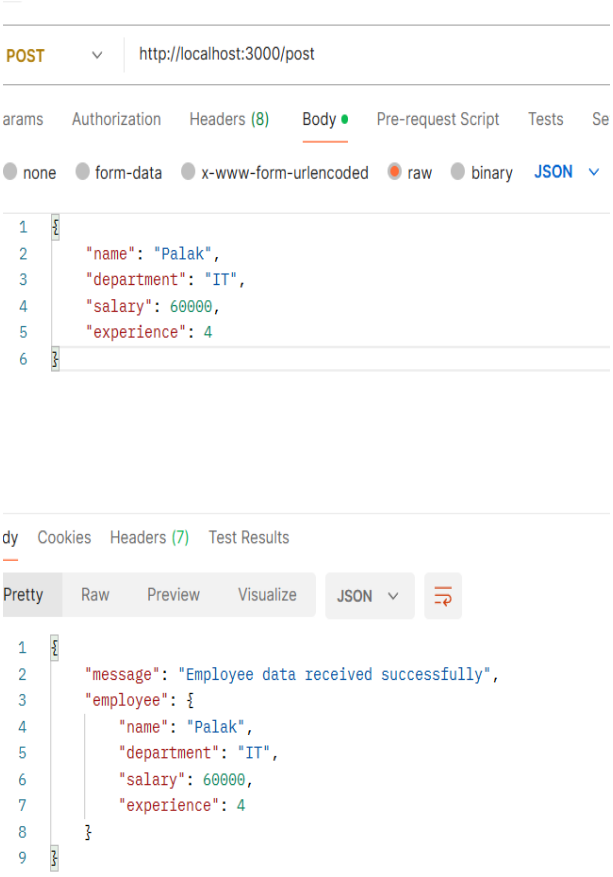
// POST route to receive JSON data
app.post("/post", (req, res) => {
    console.log("Received JSON data:", req.body);

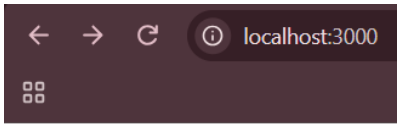
    res.json({
        message: "Employee data received successfully",
        employee: req.body
    });
});

// Start server
app.listen(3000, () => {
```

```
    console.log("Server is running on http://localhost:3000");
});
```

➢ **Output:-**





Welcome to ExpressJS

```
Method: POST
URL: /post
Time: 21/09/2026, 14:12:31
-------------------------
Received JSON data: { name: 'Palak', department: 'IT', salary: 60000, experience: 4 }
```

```
PS C:\Users\Admin\OneDrive\Desktop\Journal\Mean> node pr5.js
Server is running on http://localhost:3000
Method: GET
URL: /
Time: 21/09/2026, 14:08:02
```

Q-6 Create an ExpressJS program to serve static resources such as an HTML file, CSS, and an image using express.static().

- **Input:-**
  - **Static.js:-**

```
const express = require("express");
const path = require("path");

const app = express();

app.use(express.static(path.join(__dirname, "public")));

app.listen(3000, () => {
   console.log("Server is running on http://localhost:3000");
});
```

  - Public/index.html:-

```
<html>
<head>
   <title>Express Static Resources</title>
   <link rel="stylesheet" href="style.css">
</head>

<body>
   <h1>Welcome to ExpressJS</h1>
   <p>This HTML file is served using express.static().</p>

   <img src="img.png" alt="Sample Image" width="300">
</body>
</html>
```

  - Public/style.css:-

```
body {
   font-family: Arial, sans-serif;
   text-align: center;
   margin-top: 50px;
}

h1 {
   font-size: 35px;
}
```

```
p {
    font-size: 20px;
}

img {
    margin-top: 20px;
}
```

❖ **Public/img.png:-**



➢ **Output:-**

Welcome to ExpressJS

This HTML file is served using express.static().



Q-7 Create an ExpressJS program to develop a RESTful API for managing student records (Add, View, Update, Delete using an array).

- **Input:-**

```
const express = require("express");
const app = express();

app.use(express.json());

let students = [
  {
    id: 1,
    name: "Palak",
    age: 20,
    course: "BCA"
  },
  {
    id: 2,
    name: "Ritu",
    age: 19,
    course: "BSc IT"
  }
];

// Add Student
app.post("/students", (req, res) => {
  const student = {
    id: students.length + 1,
```

```
        name: req.body.name,
        age: req.body.age,
        course: req.body.course
    };

    students.push(student);

    res.status(201).json({
        message: "Student added successfully",
        student: student
    });
});

// View All Students
app.get("/students", (req, res) => {
    res.json(students);
});

// View Student by ID
app.get("/students/:id", (req, res) => {
    const id = parseInt(req.params.id);
    const student = students.find(s => s.id === id);

    if (!student) {
        return res.status(404).json({
            message: "Student not found"
        });
```

```
    }

    res.json(student);
});

// Update Student
app.put("/students/:id", (req, res) => {
    const id = parseInt(req.params.id);
    const student = students.find(s => s.id === id);

    if (!student) {
        return res.status(404).json({
            message: "Student not found"
        });
    }

    student.name = req.body.name;
    student.age = req.body.age;
    student.course = req.body.course;

    res.json({
        message: "Student updated successfully",
        student: student
    });
});

// Delete Student
```

```
app.delete("/students/:id", (req, res) => {
    const id = parseInt(req.params.id);
    const index = students.findIndex(s => s.id === id);

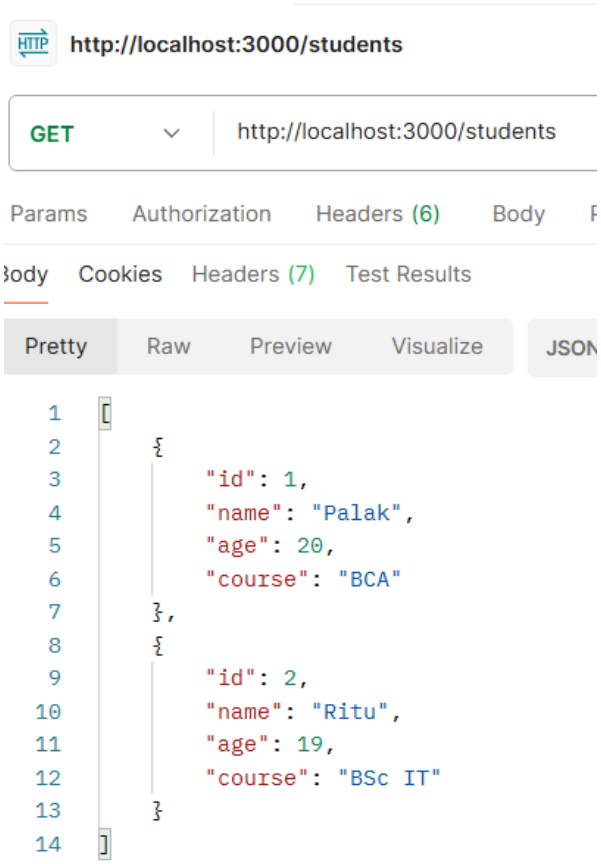
    if (index === -1) {
        return res.status(404).json({
            message: "Student not found"
        });
    }

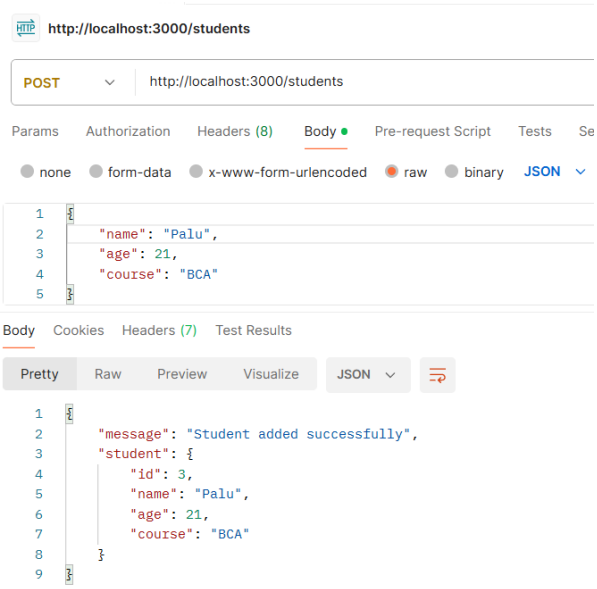
    const deletedStudent = students.splice(index, 1);

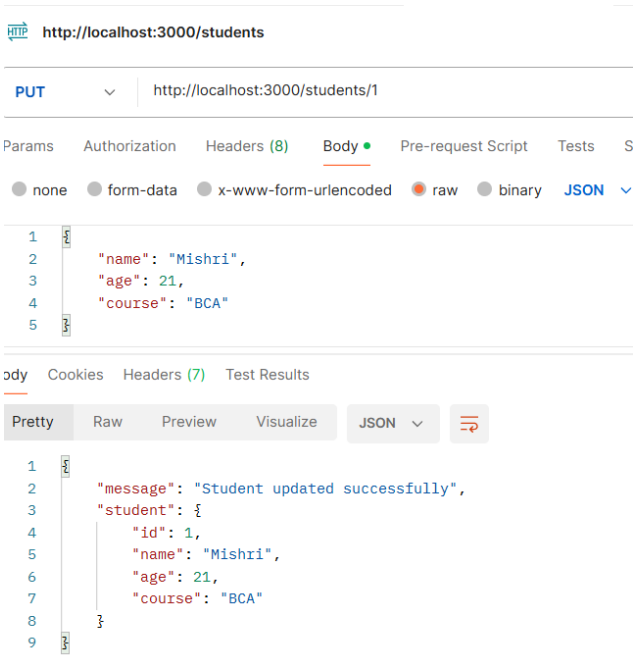
    res.json({
        message: "Student deleted successfully",
        student: deletedStudent[0]
    });
});

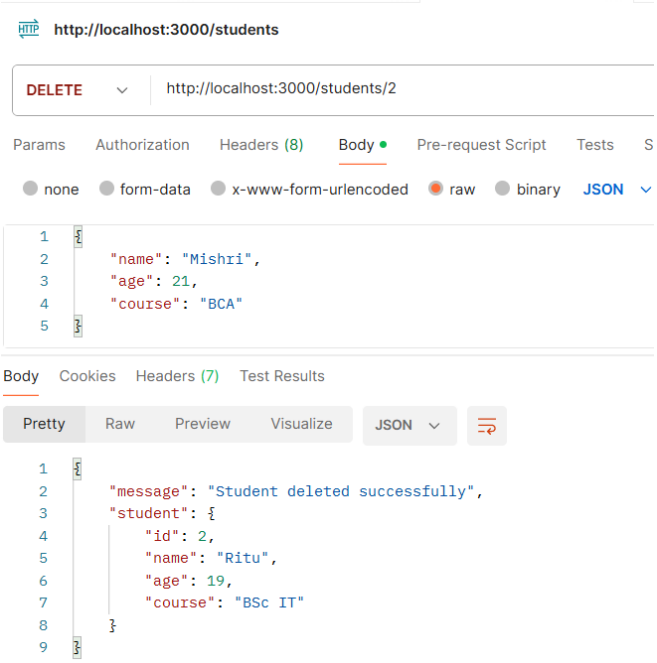
// Start Server
app.listen(3000, () => {
    console.log("Server is running on http://localhost:3000");
});
```

➢ **Output:-**


http://localhost:3000/students
GET
http://localhost:3000/students
Params
Authorization
Headers (6)
Body
Cookies
Headers (7)
Test Results
Pretty
Raw
Preview
Visualize
JSON
[
{
"id": 1,
"name": "Palak",
"age": 20,
"course": "BCA"
},
{
"id": 2,
"name": "Ritu",
"age": 19,
"course": "BSc IT"
}
]


http://localhost:3000/students
POST
http://localhost:3000/students
Params
Authorization
Headers (8)
Body
Pre-request Script
Tests
none
form-data
x-www-form-urlencoded
raw
binary
JSON
{
"name": "Palu",
"age": 21,
"course": "BCA"
}
Body
Cookies
Headers (7)
Test Results
Pretty
Raw
Preview
Visualize
JSON
{
"message": "Student added successfully",
"student": {
"id": 3,
"name": "Palu",
"age": 21,
"course": "BCA"
}
}


http://localhost:3000/students
PUT
http://localhost:3000/students/1
Params
Authorization
Headers (8)
Body
Pre-request Script
Tests
none
form-data
x-www-form-urlencoded
raw
binary
JSON
{
"name": "Mishri",
"age": 21,
"course": "BCA"
}
Cookies
Headers (7)
Test Results
Pretty
Raw
Preview
Visualize
JSON
{
"message": "Student updated successfully",
"student": {
"id": 1,
"name": "Mishri",
"age": 21,
"course": "BCA"
}
}


http://localhost:3000/students
DELETE
http://localhost:3000/students/2
Params
Authorization
Headers (8)
Body
Pre-request Script
Tests
none
form-data
x-www-form-urlencoded
raw
binary
JSON
{
"name": "Mishri",
"age": 21,
"course": "BCA"
}
Body
Cookies
Headers (7)
Test Results
Pretty
Raw
Preview
Visualize
JSON
{
"message": "Student deleted successfully",
"student": {
"id": 2,
"name": "Ritu",
"age": 19,
"course": "BSc IT"
}
}

Q-8 Create a simple REST API for a "Books" collection with endpoints:

- **GET /books → Get all books**
- **GET /books/:id → Get a book by ID**
- **POST /books → Add a new book**
- **PUT /books/:id → Update a book by ID**
- **DELETE /books/:id → Delete a book by ID**

➢ **Input:-**

❖ **Data.js:-**

```
let books = [
    {
        id: 1,
        title: "1984",
        author: "George Orwell"
    },
    {
        id: 2,
        title: "The Hobbit",
        author: "J.R.R. Tolkien"
    }
];

module.exports = books;
```

❖ **Index**.js:-

```
const express = require('express');
const app=express();
const books=require('./data');

app.use(express.json());

app.get('/',(req,res) => {
    res.send('welcome to the books API! use/API/books to get strated.');
});

app.get('/API/books',(req,res)=>{
```

```
    res.json(books);
});

app.get('/API/books/:id',(req,res)=>{
    const book=books.find(b=>b.id === parseInt(req.params.id));

    if(!book){
        return res.status(404).send('Book Not Found');
        res.json(book);
    }
});

app.post('/API/books',(req,res)=>{
    const newbook ={
        id:books.length+1,
        title:req.body.title,
        author:req.body.author
    };
    books.push(newbook);
    res.status(201).json(newbook);
});

app.put('/API/books/:id',(req,res)=>{
    const book = books.find(b=>b.id === parseInt (req.params.id));

    if(!book){
        return res.status(404).send('Book Not Found');
    }
    book.title=req.body.title;
    book.author=req.body.author;
    res.json(book);
});

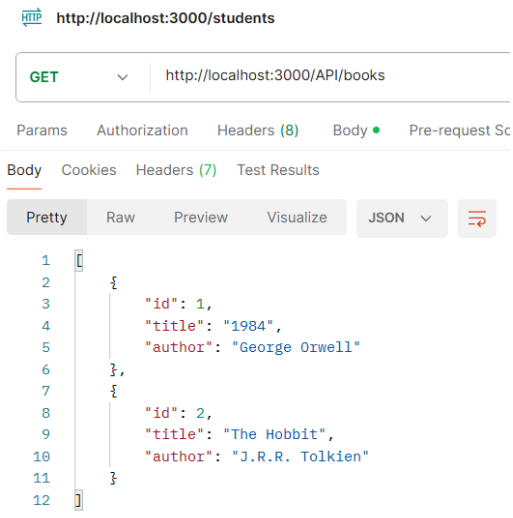
app.delete('/API/books/:id',(req,res)=>{
    const bookIndex = books.findIndex(b=>b.id === parseInt (req.params.id));

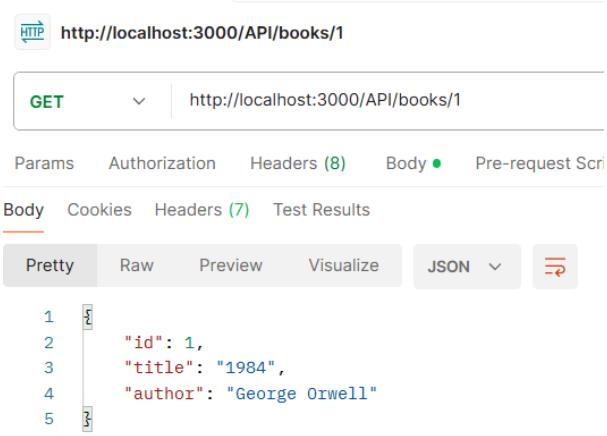
    if(bookIndex === -1)
        return res.status(404).send('Book Not Found');
    const deleteBook = books.splice(bookIndex,1);
    res.json(deleteBook[0]);
```

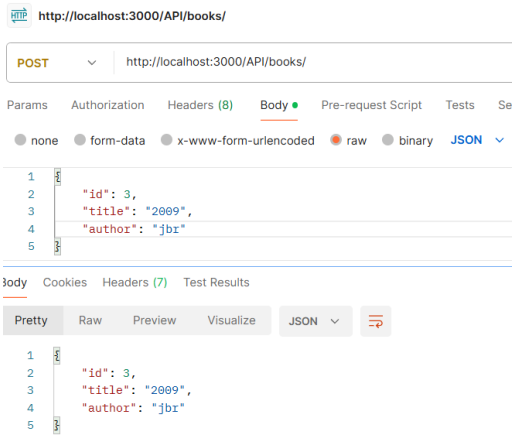
```
});

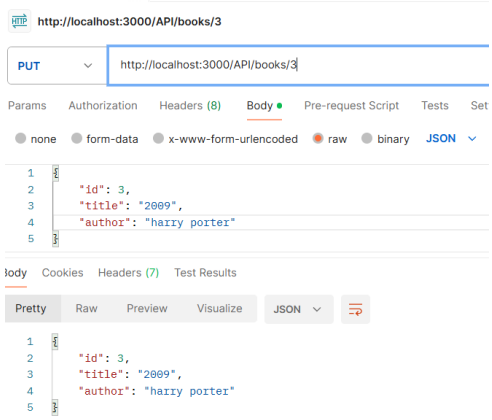
app.listen(3000,()=>{
    console.log("server running at http://localhost:3000");
});
```

- **Output:-**









Q – 9 Create a RESTful API in Node.js (Express) and connect it to MongoDB.

- **Input:-**
  - **Index.js:-**

```
const express = require("express");
const connectDB = require("./config/db");
const userRoutes = require("./routes/userRoutes");
const PORT = 3000;

const app = express();

app.use(express.json());

app.get("/", (req, res) => {
   res.send("Welcome to the Express API");
});

connectDB();

app.use("/api/users", userRoutes);

app.listen(PORT,()=>{
   console.log(`server running on http://localhost:${PORT}`);
});
```

  - **Config/db.js:-**

```
const mongoose = require("mongoose");

const connectDB = async () => {
   try {
      await mongoose.connect("mongodb://localhost:27017/studentdb");

      console.log("MongoDB connected");
   } catch (error) {
      console.log("MongoDB connection failed");
   }
};

module.exports = connectDB;
```

  - **Routes/routes.js:-**

```
const express = require("express");
const router = express.Router();

const userController = require("../controllers/userController");

router.post("/", userController.createUser);
router.get("/", userController.getUsers);
router.get("/:id", userController.getUserById);
router.put("/:id", userController.updateUser);
router.delete("/:id", userController.deleteUser);

module.exports = router;
```

- **Models/User.js:-**

```
const mongoose = require(`mongoose`);

const userSchema = new mongoose.Schema({
   name: {
      type: String,
      required: true,
   },
   email: {
      type: String,
      required: true,
   },
   age: Number,
});

module.exports = mongoose.model("User", userSchema);
```

- **Controllers/userController.js:-**

```
const User = require("../models/User");

// Create
exports.createUser = async (req, res) => {
   const user = await User.create(req.body);
   res.status(201).json(user);
};

// Read
exports.getUsers = async (req, res) => {
   const users = await User.find();
```

```
    res.json(users);
};

// Read by Id
exports.getUserById = async (req, res) => {
    const user = await User.findById(req.params.id);
    res.json(user);
};

// Update
exports.updateUser = async (req, res) => {
    const user = await User.findByIdAndUpdate(
        req.params.id,
        req.body,
        {
            new: true,
        }
    );

    res.json(user);
};

// Delete
exports.deleteUser = async (req, res) => {
    await User.findByIdAndDelete(req.params.id);

    res.json({
        message: "User deleted",
    });
};
```

➢ **Output:-**

```
PS C:\Users\Admin\OneDrive\Desktop\Journal\Mean\pr9> node index.js
server running on http://localhost:3000
MongoDB connected
```

